

AGENCYOS

PHASE 4 - UI DESIGNER AGENT

Conditional Activation & Responsibility Specification

FULL UI DESIGN + CONDITIONAL RE-ACTIVATION + REVIEW LOOPS + IMPLEMENTATION + TESTING

Official Role	UI/Figma Designer Agent
Phase	Phase 4 - Task 2
Activation Model	CONDITIONAL - activated only when a valid Phase 4 design condition requires design work/revision
Primary Mission	Expand the locked Phase 3 visual direction into the complete high-fidelity UI and re-activate only for governed QA/Admin/Client design corrections
Canonical Source	Exact approved Figma UI version / truthful configured design source
Review Order	UI Designer → Design QA → Admin → PM → Client
Hard Rule	Designer cannot self-pass QA, self-approve Admin/client gates, or silently expand scope
Prototype Boundary	After client UI approval and lock, Prototype Agent owns prototype build; Designer re-activates only if a governed UI design change is required

This document is based on the locked Phase 4 flow plus the AgencyOS UI Design & Prototype specification and preserves the existing Phase 3 Designer principles: Figma/design source of truth, limited/controlled revisions, scope traceability, version history, reuse-first behavior and no direct client bypass.

1. Core Conditional Principle

The Phase 4 UI Designer Agent is not a continuously running general-purpose agent. It becomes active when the workflow reaches a condition that legitimately requires UI design work, correction or revision. When no valid design condition exists, the agent should remain idle.

Conditional activation prevents duplicate work, uncontrolled revision cost, accidental scope expansion and stale-version edits. Every design run must have an activation reason, source version, allowed change set and expected return gate.

2. Source Basis & Locked Context

Source concept	Phase 4 application
Phase 3 UI Designer baseline	Reuse chosen theme, colors, design tokens, brand direction and canonical design source; do not restart theme exploration
Phase 4 Master Flow	Complete full UI → Design QA → Admin → Client → controlled revision loop → UI lock → prototype
UI Design & Prototype Spec	Requirement/feature/screen/state coverage, responsive states, versioning, prototype linkage and scope guardrails
QA principles	Designer does not self-certify; defects return to Designer then QA retest
Change Request rules	New material functionality does not enter as normal design revision

3. Conditional Activation Matrix

Condition	Activate UI Designer?	Required action
Phase 4 starts with valid Phase 3 baseline	YES	Create complete high-fidelity UI for approved scope
Coverage check finds a missing required screen/state	YES	Add/fix missing approved UI coverage
Business rule is ambiguous	NO - WAIT	PM/client clarification first; Designer must not guess
Design QA returns CHANGES_REQUIRED	YES	Fix exact defects; create revised UI version; return to QA
Admin returns EDIT	YES	Apply exact Admin-approved edits; return through QA before Admin again
Client requests allowed visual/design revision	YES	Apply controlled revision from PM request; return through QA + Admin
Client asks new feature/module/role/business rule	NO - NOT YET	Route to Change Request; activate only if scope change is approved
Client approves exact UI version	NO	UI locks; Prototype Agent becomes primary build owner
Prototype QA finds code/interaction defect only	NO	Prototype Agent fixes; Designer not required
Prototype QA identifies true source-UI design ambiguity/error	CONDITIONAL YES	Re-activate only after structured classification confirms UI design change is required
Admin prototype review requests only build behavior fix	NO	Prototype Agent handles
Admin/client requests UI redesign after UI lock	CONDITIONAL / ESCALATE	Create governed revision/change decision; never overwrite locked UI
Duplicate/replayed event for same design job	NO NEW WORK	Return existing artifact/job result idempotently

4. Initial Phase 4 Activation - Full UI Design

- ☐ Consume the exact locked Phase 3 theme and color selection.
- ☐ Consume approved scope / requirement version.
- ☐ Consume the final screen list and screen-by-screen content/action baseline.
- ☐ Consume Project Planning context and client references.
- ☐ Run/consume coverage check before designing.
- ☐ Design every required screen for the approved product.
- ☐ Design required role variants.
- ☐ Design applicable loading, empty, validation, error, success, permission and offline/network states.
- ☐ Design responsive/device/platform variants required by scope.
- ☐ Reuse Phase 3 tokens/components/assets instead of recreating the visual system.
- ☐ Store exact Figma/design references and version identity.

5. Phase 4 Designer Inputs

- ☐ project_id / organization_id.
- ☐ active scope / requirement version.
- ☐ screen inventory.
- ☐ screen content/action baseline.
- ☐ user roles/personas.
- ☐ critical flows.
- ☐ Phase 3 selected theme ID and color ID.
- ☐ locked Phase 3 Figma/design version.
- ☐ brand assets and client references.
- ☐ accessibility/device/platform requirements.
- ☐ current UI version and revision history.
- ☐ activation condition + allowed change set.

6. Complete High-Fidelity UI Responsibilities

6.1 Requirement-to-Screen Traceability

- ☐ Every designed screen maps to approved feature/requirement or an explicit shared/system UI need.
- ☐ Do not leave required features with no UI representation where UI is needed.
- ☐ Do not add orphan screens without approved rationale.

6.2 Screen Record Completeness

- ☐ Screen name/ID, role, purpose, entry/exit, data, actions, validation, states, responsive behavior and version must be represented in the design/system record where applicable.

6.3 Design States

- ☐ Default, hover/focus where applicable, pressed/active, disabled, loading, empty, validation error, system error, success, permission denied, offline/network failure where applicable, long-content/overflow.

6.4 Responsive / Platform Variants

- ☐ Create only variants required by the approved platform scope.
- ☐ Ensure text wrapping, tables/lists, navigation adaptation, touch targets and responsive behavior are coherent.

6.5 Reusable Design System

- ☐ Reuse Phase 3 tokens/components.
- ☐ Extend reusable components only where full UI requires new patterns.
- ☐ Avoid one-off styling when a reusable primitive is appropriate.
- ☐ Do not create a second conflicting design system.

6.6 Developer/Prototype Readiness

- ☐ Use structured layer/component naming where possible.
- ☐ Keep exact screen/version references stable.
- ☐ Make critical flows understandable for Prototype Agent and later Phase 5 development.

7. Design QA Conditional Re-Activation

- ☐ Activation trigger must reference exact UI version and exact QA defect(s).
- ☐ Designer fixes only the defect plus necessary impact area.
- ☐ Designer does not silently change unrelated approved screens.
- ☐ A revised UI version is created when design content changes.
- ☐ Change summary records what changed and why.
- ☐ Revised version returns to Design QA.
- ☐ Designer cannot set QA_PASS.

QA CHANGES_REQUIRED → Designer revision → QA retest. The agent saying 'fixed' changes status to FIX_READY / revision-ready, not VERIFIED.

8. Admin EDIT Conditional Re-Activation

- ☐ Admin decision identifies exact UI version under review.
- ☐ Admin comments/reason are stored as structured revision input.
- ☐ Designer applies approved edit to the correct source version.
- ☐ Designer creates new/revised version and change summary.
- ☐ Internal/Design QA runs again.
- ☐ Only QA-passed revised UI returns to Admin.
- ☐ Designer does not send directly to client.

Locked loop: ADMIN EDIT → UI DESIGNER → DESIGN QA → ADMIN AGAIN.

9. Client Revision Conditional Re-Activation

- ☐ Client feedback reaches Designer only through PM/workflow.
- ☐ Feedback references the exact UI version the client saw.
- ☐ PM/workflow classification confirms it is an allowed design revision, not unresolved scope.
- ☐ Designer applies only the approved change set.
- ☐ Revision origin is CLIENT.
- ☐ New/revised UI version is created.
- ☐ Design QA validates the revision.
- ☐ Admin re-approves before PM re-shares.
- ☐ Revision count is updated.

Locked loop: CLIENT FEEDBACK → PM CLASSIFICATION → DESIGNER → QA → ADMIN → PM RE-SHARE → CLIENT.

10. Conditions Where Designer Must NOT Activate

- ☐ Missing/ambiguous business rule - wait for clarification.
- ☐ New feature/module/role/business logic not yet approved through scope/change governance.
- ☐ Prototype code bug with no source-UI design change.
- ☐ Payment/invoice issue.
- ☐ WhatsApp/message delivery issue.
- ☐ Admin/client approval task.
- ☐ QA verification task.
- ☐ Phase 5 production logic implementation.
- ☐ Duplicate retry for same unchanged input/version.
- ☐ Pure data/backend/API issue that does not change approved UI.

11. Scope Protection Rules

Incoming request	Designer action
Spacing/color/component correction within approved UI	Implement as controlled design revision
Missing UI state already required by scope	Implement correction
New screen needed for already-approved feature that coverage missed	Implement after coverage/workflow confirms it is required
New feature not in approved scope	Do not design yet; route to Change Request
Client wants complete new visual direction after approval	Escalate; treat as material revision/change according to policy
Prototype implementation differs from approved UI	Designer changes only if approved UI itself is wrong; otherwise Prototype Agent fixes implementation

12. UI Versioning Rules

- ☐ Every meaningful design revision has a unique UI version identity.
- ☐ Store parent/from-version when applicable.
- ☐ Store activation reason: INITIAL / QA / ADMIN / CLIENT / APPROVED_SCOPE_CHANGE.
- ☐ Store source scope version.
- ☐ Store changed screens/components.
- ☐ Store change summary.
- ☐ Store QA/Admin/client decisions tied to exact version.
- ☐ Never overwrite an approved/locked version silently.
- ☐ Alternative/historical versions remain accessible.

13. Figma / Canonical Design Source Rules

- ☐ Treat the exact configured Figma/design artifact as canonical source.
- ☐ Store stable file/page/node references where supported.
- ☐ Preview PNG/JPG/PDF is secondary evidence, not the canonical editable source.
- ☐ Do not silently replace a Figma node/file while reusing the same version ID.
- ☐ If Figma write automation is unavailable, expose a truthful manual-assisted action state.
- ☐ Never claim a Figma change was written if only a preview/mockup was generated.
- ☐ Keep provider credentials/tokens in secure configuration.

14. Image Generation Conditional Use

- ☐ Use image generation only when it adds genuine value: mood/reference/illustration/background/supporting asset.
- ☐ Do not use generated screenshots as a substitute for canonical app UI design.
- ☐ Do not regenerate an asset on every revision if unchanged.
- ☐ Generated assets must be linked to the relevant design version and usage purpose.
- ☐ Client-facing asset must not contain internal prompts/provider metadata.

15. Handoff to Prototype Agent

- ☐ UI version is explicitly client-approved.
- ☐ UI version is locked.
- ☐ Complete screen inventory is present.
- ☐ Critical flows and navigation relationships are clear.
- ☐ Required design states are represented.
- ☐ Tokens/components/assets are identifiable.
- ☐ Known limitations are documented.
- ☐ Prototype Agent receives exact ui version id rather than 'latest'

After valid UI lock, the UI Designer becomes inactive by default. Prototype Agent becomes the primary Phase 4 execution agent until a true UI design change condition reactivates Designer.

16. Prototype-Stage Conditional Designer Re-Activation

Prototype-stage finding	Designer?	Reason
Button route broken but approved UI is correct	NO	Prototype implementation defect
Text overlaps because prototype CSS/layout is wrong	NO initially	Prototype Agent should match approved UI; Designer only if approved design itself is impossible/ambiguous
Approved UI has no error state required by scope	YES	Source UI coverage defect
Prototype reveals a genuine usability conflict in approved UI	CONDITIONAL	Requires structured design-change decision; new UI version + QA/Admin/client as applicable
Client asks different color after UI lock	CONDITIONAL / ESCALATE	Material approved-design change; do not silently edit locked baseline
Client asks new flow	NO until scope approval	Scope/change request first

17. Designer Agent Must NOT

- ☐ Must not communicate directly with client when PM owns communication.
- ☐ Must not mark Design QA PASS.
- ☐ Must not mark Admin approval.
- ☐ Must not mark client approval.
- ☐ Must not start Prototype build.
- ☐ Must not verify Finance/payment.
- ☐ Must not add material product functionality without scope/change approval.
- ☐ Must not overwrite approved/locked UI.
- ☐ Must not restart theme/color exploration during Phase 4 without governed material-change decision.
- ☐ Must not fabricate missing requirements.
- ☐ Must not expose internal AI/provider/model/API information in client-facing artifacts.
- ☐ Must not create duplicate designs on replay of the same job/input version.

18. Admin Panel - Conditional Designer Visibility

Admin surface	Required visibility
UI Design Overview	current UI version, activation reason, status, owner, blocker, next gate
Phase 3 Baseline	locked theme/colors/source version being reused
Coverage Matrix	requirement/feature/screen/state mapping and missing items
UI Versions	version lineage, activation reason, changed screens, status
Design QA	defects, fix-ready, retest and exact PASS version
Admin Review	exact version, Confirm/Edit, comments and decision history
Client Review	exact version shared, feedback classification and revision count
Figma / Design Source	file/page/node refs, availability/write/manual-assisted status
Activity / Audit	every activation, revision, state change and approval relationship

19. Designer-Specific Data Contracts

Entity	Minimum fields
UIDesignJob	id, project/organization, activation_reason, source_ui_version, scope_version, allowed_change_set, status, idempotency_key
UIVersion	id/version, project, parent_version, source_phase3_version, scope_version, Figma/design refs, status, lock state
UIScreenVersion	ui_version, screen_id, role, states/variants, design node refs, status
UIDesignRevision	from/to version, origin QA/ADMIN/CLIENT/SCOPE, request/evidence, affected screens, change summary
UIDesignBlocker	job/version, blocker type, owner, reason, clarification needed, resume condition
DesignQADefectLink	ui_version, defect_id, affected screen/component, fix version, retest status
FigmaArtifactRef	file/page/node/version, preview refs, write/read capability status
AuditEvent	actor/agent, action, version/screen, before/after, project/tenant, timestamp

20. Conditional Event Contract

Event	Designer behavior
Phase4Started	Activate INITIAL full-UI job if valid baseline
UICoverageGapConfirmed	Activate only for approved missing screen/state coverage
DesignQACHangesRequired	Activate QA correction job for exact version/defects
AdminUIEditRequested	Activate Admin revision job for exact version
ClientUIChangeApproved	Activate client revision job for exact version
ScopeChangeApprovedForUI	Activate only for approved new/changed UI scope
ClientUIApproved	Do not create new design job; lock UI and hand to Prototype
PrototypeDesignIssueConfirmed	Conditional re-activation only when source UI design truly requires change
Duplicate event replay	Return existing job/artifact; no duplicate design generation

21. Idempotency & Duplicate-Work Rules

- ☐ Design job idempotency key must include project + activation type + source version + normalized change request where appropriate.
- ☐ Replay of same unchanged job returns/reuses existing result.
- ☐ QA retry does not create another unrelated design version.
- ☐ Admin/client revision creates a new version only if design content actually changes.
- ☐ No duplicate Figma page/file is created simply because delivery/event retry occurred.
- ☐ Artifact generation and workflow transition are decoupled from message retry.

22. Security & Permission Boundaries

- ☐ Designer can create/update authorized design artifacts only for its project/tenant.
- ☐ Designer cannot grant itself higher autonomy/permissions.
- ☐ Designer cannot approve Admin/client gates.
- ☐ Designer cannot access unrelated project assets/Figma refs.
- ☐ Design/provider secrets remain server-side.
- ☐ Client-facing exports omit internal prompts, API keys and private agency metadata.
- ☐ Sensitive state changes and lock/unlock operations are audited.

23. Cost-Control Rules

- ☐ Reuse Phase 3 tokens/components/assets and prior valid screen patterns.
- ☐ Initial full UI run designs only approved scope.
- ☐ QA/Admin/client correction should target affected screens rather than regenerate entire UI when unnecessary.
- ☐ Do not generate multiple theme directions again in Phase 4 unless a governed material-design reset is approved.
- ☐ Use deterministic tools/scripts for mechanical Figma/layout operations where appropriate.
- ☐ Use expensive model reasoning only for genuinely ambiguous design reasoning.
- ☐ Track agent/model/tool usage where AgencyOS telemetry supports it.
- ☐ Revision limits prevent uncontrolled client design cost.

24. Failure & Recovery Handling

Failure	Required designer behavior
Phase 3 baseline missing	Block; do not redesign from assumptions
Business rule ambiguous	Create clarification blocker; PM/client resolves
Figma read unavailable	Use stored approved metadata if sufficient or block; do not guess
Figma write unavailable	Use manual-assisted write step; no fake success
Asset missing	Request/flag missing asset or use approved fallback
Design job crashes	Persist job/version state; safe retry
Duplicate event	No duplicate design version
QA defect arrives for stale UI version	Reject/route to current correct version; do not patch blindly
Admin/client change conflicts with locked scope	Escalate to scope/change governance

25. Technical Implementation Plan

UID4-I01 Existing Designer Discovery

Inspect current ui_designer agent, Phase 3 outputs, Figma integration, screen/domain model, QA loop, Admin/client review and prototype handoff.

UID4-I02 Conditional Activation Engine

Implement explicit activation reasons and eligibility rules; no generic always-on designer execution.

UID4-I03 Input Contract

Require source UI/scope versions, screen coverage, design baseline and allowed change set.

UID4-I04 UI Version Domain

Implement/extend immutable version lineage, parent links, activation reason and exact design refs.

UID4-I05 Full UI Job

Implement initial Phase 4 complete high-fidelity design path from locked Phase 3 baseline.

UID4-I06 Screen / State Coverage

Implement requirement→screen→state mapping and missing/duplicate/orphan checks.

UID4-I07 Responsive / Platform Variants

Generate only scope-required device/platform variants.

UID4-I08 Design QA Revision Path

QA defect → targeted Designer revision → new version → QA retest.

UID4-I09 Admin Edit Path

Admin edit → targeted revision → QA → Admin.

UID4-I10 Client Revision Path

PM-classified allowed client revision → Designer → QA → Admin → PM.

UID4-I11 Scope-Change Guard

Block new material functionality until approved change request reaches UI workflow.

UID4-I12 Figma Capability Truthfulness

Detect read/write/create capability; use manual-assisted state when write is unavailable.

UID4-I13 UI Lock & Prototype Handoff

After explicit client approval, lock exact UI and stop Designer by default.

UID4-I14 Prototype-Stage Re-Activation

Implement governed re-activation only for true source-design defects/approved design changes.

UID4-I15 Admin Visibility

Expose activation reason, job state, versions, coverage, QA, reviews, Figma refs and blockers.

UID4-I16 Reliability / Idempotency

Prevent duplicate design jobs/artifacts on event/message retry.

UID4-I17 Security / Permissions

Project/tenant scope, secret protection, approval boundaries and audit.

UID4-I18 Testing / Regression

Test all activation branches, no-activation branches, revisions, scope guard and handoff.

26. Conditional Testing Matrix

Area	Required coverage
Initial activation	valid Phase 3 baseline; invalid/missing baseline; duplicate start
Coverage activation	confirmed missing required UI vs false/duplicate gap
Clarification	ambiguous rule must block Designer instead of guessing
QA activation	exact defects/version; targeted fix; QA retest
Admin activation	exact edit/version; mandatory QA before Admin again
Client activation	PM-classified allowed revision; exact client-shared version
Scope guard	new feature does not activate design until change approved
UI lock	client-approved exact version stops normal Designer activity
Prototype-stage branch	code bug does not activate Designer; true source-UI defect may
Figma	read/write available vs missing; manual-assisted state
Versioning	parent/from-version, changed screens, historical versions
Idempotency	duplicate events/retries do not create duplicate design work
Security	cross-project/tenant access and approval privilege denied
Admin UI	activation reason/job/version/blocker/history visible
Regression	Phase 3 locked direction and Phase 4 prototype handoff remain intact

27. Mandatory Detailed Test Cases

ID	Scenario	Expected result
UID4-T001	Valid Phase 4 start	Designer activates INITIAL full-UI job
UID4-T002	Phase 3 incomplete	Designer does not activate; blocker
UID4-T003	Theme/color not locked	Designer does not restart theme design; blocker
UID4-T004	Duplicate Phase4Started event	No duplicate full-UI job
UID4-T005	Approved requirement lacks required screen	Coverage-confirmed activation creates needed UI
UID4-T006	Business rule ambiguous	Designer does not guess; clarification required
UID4-T007	Create complete UI from locked visual system	All required screens/states map to scope
UID4-T008	Designer attempts new feature	Blocked/routed to Change Request
UID4-T009	Design QA returns defect	Designer activates QA correction job
UID4-T010	QA defect references stale version	Do not patch blindly; resolve version mismatch
UID4-T011	Designer marks own QA PASS	Denied
UID4-T012	Admin EDIT exact version	Designer revises and returns to QA
UID4-T013	Try send Admin-edit revision directly to client	Blocked
UID4-T014	Client allowed visual revision	Designer activates only from PM/workflow request
UID4-T015	Client asks new module	No design activation until scope approval
UID4-T016	Revision limit exceeded	No uncontrolled design loop; escalation
UID4-T017	Client approves exact UI	Designer normal activity stops; UI locks
UID4-T018	Prototype route bug only	Designer remains inactive
UID4-T019	Prototype reveals missing source-UI error state	Conditional Designer re-activation
UID4-T020	Prototype suggests source-UI usability change after lock	Governed revision/change required
UID4-T021	Figma write unavailable	Manual-assisted state; no fake write
UID4-T022	Replay same QA correction event	No duplicate design version/job
UID4-T023	Small revision	Only affected screens/components change where feasible

ID	Scenario	Expected result
UID4-T024	Approved version overwritten in place	Rejected; new version required
UID4-T025	Cross-project Figma/design access	Denied
UID4-T026	Designer attempts Admin approval	Denied
UID4-T027	Designer attempts client approval	Denied
UID4-T028	Designer attempts payment verification	Denied
UID4-T029	Client-facing export leaks provider/model details	Security/policy FAIL
UID4-T030	Prototype handoff uses 'latest' instead of exact locked version	FAIL; exact ui_version_id required

28. Mandatory Conditional E2E Scenario

- ☐ Receive valid Phase 3 completion and activate initial Phase 4 Designer job.
- ☐ Load locked theme/colors/screen baseline/scope.
- ☐ Coverage check identifies one genuinely missing required error state; Designer adds it.
- ☐ Designer creates complete UI version.
- ☐ Design QA finds two defects.
- ☐ Designer activates QA-correction job, fixes only affected areas and creates revised UI version.
- ☐ QA retests and passes.
- ☐ Admin requests one edit.
- ☐ Designer activates Admin-edit job, revises, returns to QA, then Admin re-review.
- ☐ PM shares exact Admin-approved UI version to client.
- ☐ Client asks one allowed visual revision.
- ☐ Designer activates client-revision job only after PM classification.
- ☐ QA retests; Admin re-approves; PM re-shares.
- ☐ Client explicitly approves exact UI version.
- ☐ UI locks; Designer becomes inactive by default.
- ☐ Prototype Agent builds exact locked UI.
- ☐ Prototype QA finds one routing/code defect - Designer remains inactive.
- ☐ Later prototype QA identifies a true missing approved UI state - workflow classifies source-design defect.
- ☐ Designer conditionally re-activates, creates new governed UI version, passes QA/Admin/client flow as required.
- ☐ Final exact UI version re-locks and Prototype Agent receives exact version ID.
- ☐ All activations, non-activations, versions and reasons are visible in Admin history.

29. Definition of Done - Conditional UI Designer Agent

- ☐ Designer activation is condition-driven, not always-on.
- ☐ Initial Phase 4 full-UI design uses exact locked Phase 3 baseline.
- ☐ Every required screen/state maps to approved scope.
- ☐ Designer does not guess ambiguous business rules.
- ☐ QA changes re-activate Designer correctly.
- ☐ Admin EDIT re-activates Designer and forces QA before Admin again.
- ☐ Allowed client revisions re-activate Designer through PM/workflow.
- ☐ New scope does not enter design before Change Request approval.
- ☐ Every meaningful revision creates/links exact version history.
- ☐ Figma/design source is truthful and traceable.
- ☐ Client-approved UI locks and normally deactivates Designer.
- ☐ Prototype-stage code defects do not unnecessarily reactivate Designer.
- ☐ True source-UI design defects can reactivate Designer through governed conditions.
- ☐ Duplicate events/retries do not duplicate design work.
- ☐ Admin Panel shows activation reason, current version, QA/Admin/client history and blockers.
- ☐ Security/project/tenant boundaries pass.
- ☐ No unresolved UI Designer Phase 4 P0/P1 defects remain.

30. Master Conditional Checklist

A. Activation Guard

- ☐ activation reason
- ☐ source version
- ☐ scope version
- ☐ allowed change set
- ☐ idempotency key

B. Initial Full UI

- ☐ locked Phase 3 baseline
- ☐ all required screens
- ☐ states
- ☐ roles
- ☐ responsive/platform variants
- ☐ tokens/components

C. Coverage

- ☐ requirement mapping
- ☐ screen mapping
- ☐ state mapping
- ☐ duplicate/orphan checks
- ☐ clarifications

D. QA Re-Activation

- ☐ exact defect
- ☐ affected screens
- ☐ new version
- ☐ change summary
- ☐ QA retest

E. Admin Re-Activation

- ☐ exact Admin edit
- ☐ revision
- ☐ QA
- ☐ Admin re-review

F. Client Re-Activation

- ☐ PM classification
- ☐ allowed revision
- ☐ revision count
- ☐ QA
- ☐ Admin
- ☐ PM re-share

G. Scope Guard

- ☐ new feature detection
- ☐ Change Request route
- ☐ no silent design expansion

H. Figma / Design Source

- ☐ file/page/node/version
- ☐ preview refs
- ☐ read/write capability
- ☐ manual-assisted state

I. UI Lock

- ☐ explicit client approval
- ☐ exact version
- ☐ locked status
- ☐ historical versions

J. Prototype Boundary

- ☐ Designer inactive by default
- ☐ code bug stays with Prototype Agent
- ☐ true design defect conditional reactivation

K. Admin Panel

- ☐ activation reason
- ☐ versions
- ☐ coverage
- ☐ QA
- ☐ Admin/client reviews
- ☐ blockers
- ☐ audit

L. Reliability

- ☐ idempotency
- ☐ retry
- ☐ duplicate event
- ☐ safe recovery

M. Security

- ☐ tenant/project scope
- ☐ secrets
- ☐ no QA/Admin/client/payment privilege

N. Testing

- ☐ unit
- ☐ integration
- ☐ event/job
- ☐ branch tests
- ☐ security
- ☐ negative
- ☐ E2E
- ☐ regression

31. Final Locked Conditional Flow

PHASE 4 START + VALID BASELINE → ACTIVATE DESIGNER FOR FULL UI → COVERAGE / FULL DESIGN → DESIGN QA → QA DEFECT? RE-ACTIVATE DESIGNER → QA RETEST → ADMIN → ADMIN EDIT? RE-ACTIVATE DESIGNER → QA → ADMIN → PM/CLIENT → CLIENT ALLOWED REVISION? RE-ACTIVATE DESIGNER → QA → ADMIN → PM/CLIENT → CLIENT APPROVES EXACT UI → LOCK UI → DESIGNER INACTIVE → PROTOTYPE AGENT → IF PROTOTYPE CODE DEFECT: DESIGNER STAYS INACTIVE → IF TRUE SOURCE-UI DESIGN DEFECT /

Locked conditional principle: UI Designer Agent should run only when the workflow proves design work is required. Every activation must be traceable to a valid condition, exact source version and governed change set; otherwise the agent remains inactive.